

Language Models for GeoAI

Intro to Language Models

Etzion Harari



Part 1 — Introduction to NLP

What is NLP? Tasks, challenges & latent space



Part 2 — From Text to Vectors

How we can represent text as a vector?



Part 3 — Language Models

Transformers 101

Introduction to NLP

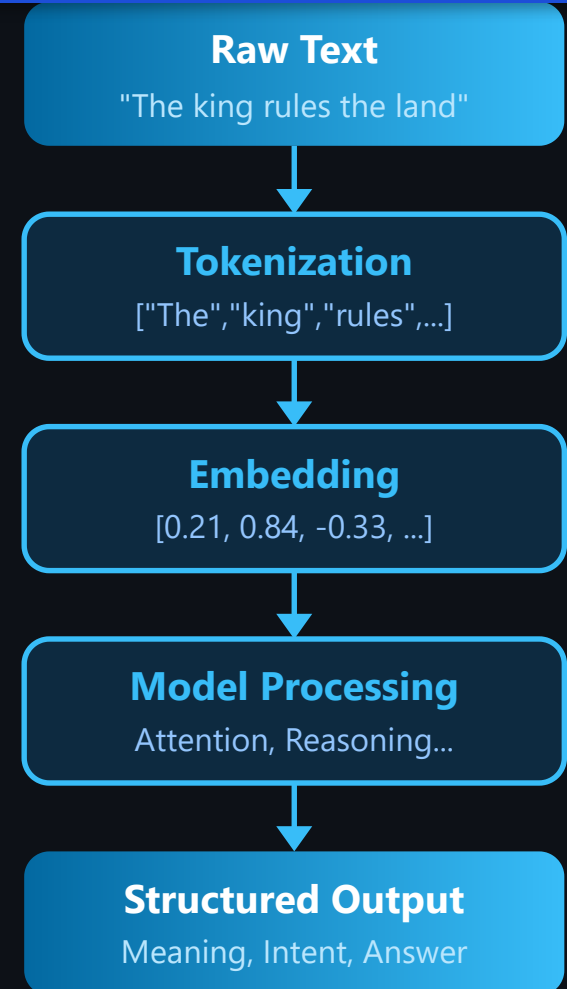
Understanding Natural Language Processing — what machines need to learn about human language

DEFINITION

NLP is the field of AI that enables computers to **understand, interpret, and generate** human language in a way that is meaningful and useful.

CORE GOALS

- 1 **Understanding** — parse meaning from text (syntax & semantics)
- 2 **Ambiguity resolution** — same word, different meanings
- 3 **Context awareness** — meaning depends on surrounding words
- 4 **Generation** — produce coherent, relevant text





Machine Translation

"Bonjour monde" → "Hello world"



Sentiment Analysis

"Loved the product!" → Positive ✓



Named Entity Recognition

[Apple]_{ORG} founded by [Jobs]_{PERSON}



Question Answering

"Who wrote Hamlet?" → "Shakespeare"



Summarization

Long article → concise 3-sentence summary



Text Generation

Prompt → coherent, contextual response

⚡ Lexical Ambiguity

🏠 "I went to the **bank**" → financial or river?

💛 "I need a **hand**" → body part or help?

⚡ "The **current**" → water or electricity?

🔗 Coreference

"Hot in **Paris**. The **city** was crowded."

→ "city" = "Paris" — requires world knowledge

🌐 Context Dependency

"Fly to **Paris**" vs "**Paris** Hilton"

→ Same word, completely different meaning

📐 Syntax ≠ Semantics

"Colorless green ideas sleep furiously"

→ Grammatically correct, semantically nonsense!

💡 The Solution

We need representations that capture **meaning in context**

→ **latent space representations**

What is a Latent Space?

Imagine giving every word a **location on a map**. Words with similar meanings are placed close together; unrelated words are far apart. That "map" is the latent space — a mathematical coordinate system where position = meaning.

WHAT ARE "LATENT COORDINATES"?

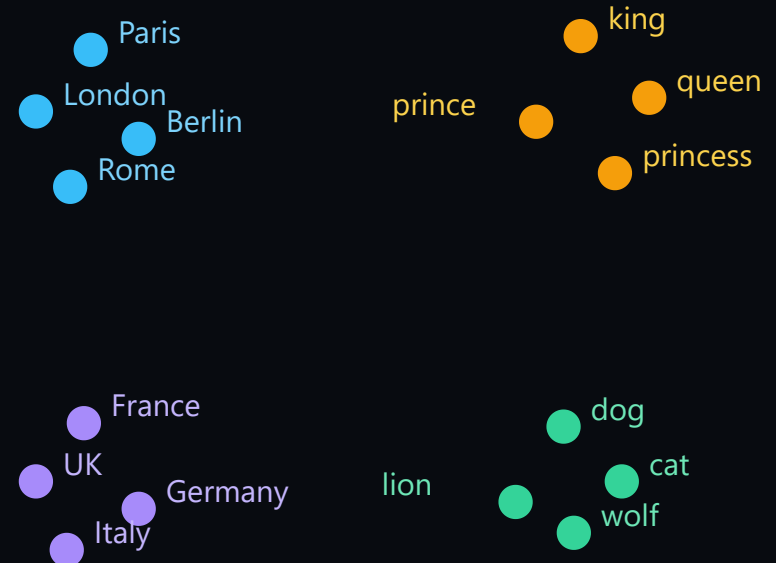
A word is assigned a list of numbers (e.g. 300 numbers). These numbers are the word's **coordinates** in the latent space — its hidden (*latent*) fingerprint of meaning.

Just like GPS coordinates locate a city on Earth, latent coordinates locate a word in meaning-space.

KEY PROPERTIES

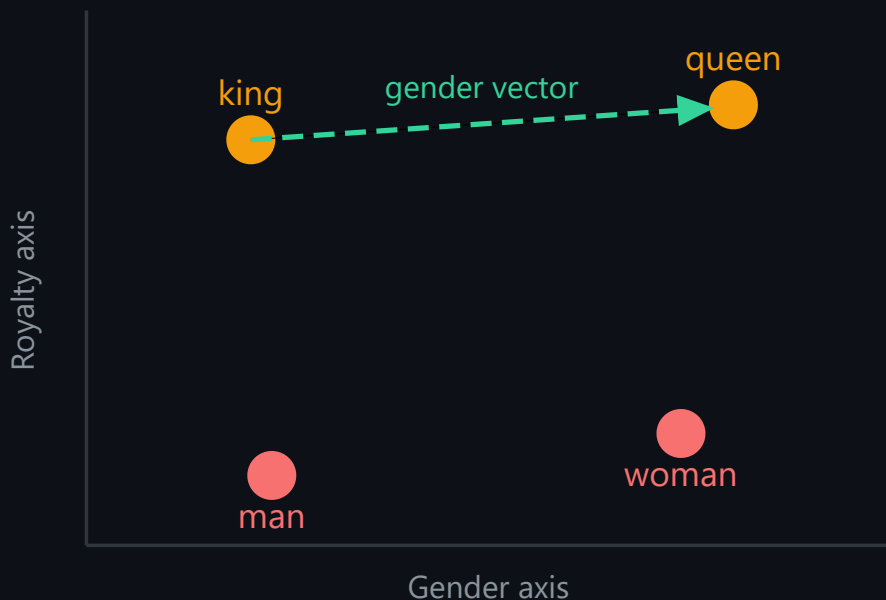
- ♦ Semantic similarity = geometric closeness
- ♦ Relationships preserved as vector directions
- ♦ Analogies = vector arithmetic

Semantic Dimension 2



Semantic Dimension 1

$$[\text{king}] - [\text{man}] + [\text{woman}] \approx [\text{queen}]$$



MORE EXAMPLES

$$[\text{France}] - [\text{Paris}] + [\text{Berlin}] \approx [\text{Germany}]$$

$$[\text{walking}] - [\text{walk}] + [\text{swim}] \approx [\text{swimming}]$$

$$[\text{doctor}] - [\text{man}] + [\text{woman}] \approx [\text{nurse}]$$



Why This Matters

The vector space **encodes meaning as geometry**.
Mathematical operations on vectors = reasoning about language and concepts.

The Formula

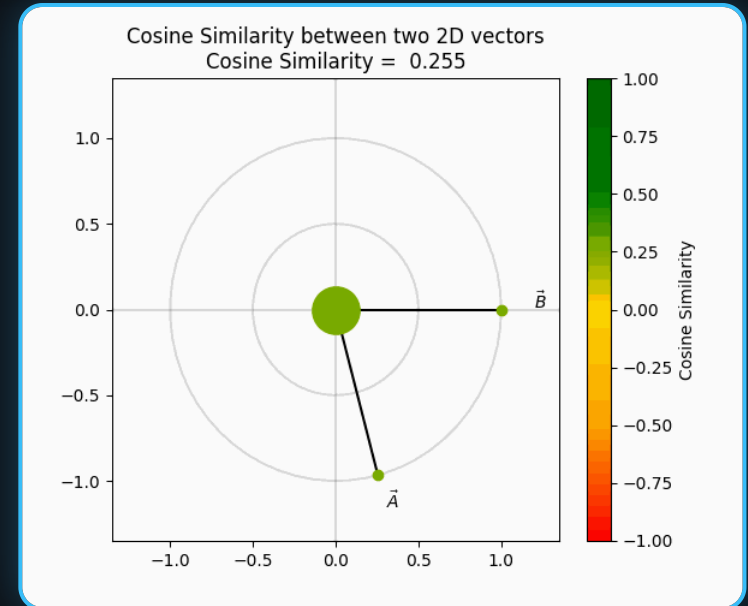
$$\cos(\theta) = (\vec{A} \cdot \vec{B}) / (\|\vec{A}\| \cdot \|\vec{B}\|)$$

Measures the **angle** between two vectors — direction similarity, not magnitude.

INTERPRETATION

- +1** Same direction → **identical meaning**
- 0** Perpendicular → **unrelated concepts**
- 1** Opposite direction → **antonyms**

```
sim(king, queen) ≈ 0.91  
sim(Paris, France) ≈ 0.88  
sim(king, fish) ≈ 0.12
```



Cosine similarity — angle between vectors matters, not length

FROM TEXT TO VECTORS

Evolution of text representation — from simple word counts to
contextual neural embeddings

The Core Problem

Computers understand **numbers**, not words. We must convert text to numerical vectors that preserve *meaning, context, and relationships*.

REQUIREMENTS

- 1 **Semantic** — similar words → similar vectors
- 2 **Contextual** — word meaning changes with context
- 3 **Relational** — encode word-to-word relationships
- 4 **Dense** — compact, not sparse one-hot vectors

Evolution of Methods

← Better Context

- 1 **Bag of Words**
Word counts only
- 2 **TF-IDF**
Weighted word importance
- 3 **Word2Vec**
Neural embeddings (2013)
- 4 **LSTM**
Sequential context (2014+)
- 5 **Transformers** ⚡
Full context → Part 3!

How It Works

Represent a document as a **vector of word counts** over a fixed vocabulary.
Vocabulary size = vector dimensions.

Document	the	king	rules	land	queen	castle
----------	-----	------	-------	------	-------	--------

Doc 1: "The king rules the land"	2	1	1	1	0	0
----------------------------------	---	---	---	---	---	---

Doc 2: "The queen rules the castle"	2	0	1	0	1	1
-------------------------------------	---	---	---	---	---	---

Doc 1: [2, 1, 1, 1, 0, 0]

Doc 2: [2, 0, 1, 0, 1, 1]

⚠ No word order — "dog bites man" = "man bites dog"

⚠ No semantic similarity — "king" ≠ "ruler"

⚠ Sparse, huge vectors (vocab ≈ 50k+ dims)

⚠ Common words dominate (the, a, is...)

⚠ No context — one meaning per word

✓ **Simple, fast, interpretable.** Good baseline for document classification.

The Idea

A word is important if it appears **frequently in this document** but **rarely across all documents**. Downweights common words like "the".

t = term | d = document | N = corpus size

$$\frac{\text{freq}(t, d)}{|\text{words in } d|} \times \frac{\log(N / \text{df}(t))}{\text{IDF}(t)} = \text{TF-IDF}(t, d)$$

Worked Example

Doc: "Paris is the capital of France" (6 words)

Corpus: N=100 docs; "Paris" in 10 of them

$$\text{TF} = 1/6 \approx 0.167$$

$$\text{IDF} = \log(100/10) = \log(10) \approx 2.30$$

$$\text{TF-IDF} = 0.167 \times 2.30 \approx 0.38$$

✓ **Improves over BoW** — reduces noise from common words. Great for search & retrieval.

"the"

TF: HIGH | IDF: Very Low (everywhere)



"earthquake"

TF: MEDIUM | IDF: HIGH (rare, specific)



"Paris"

TF: MEDIUM | IDF: MEDIUM

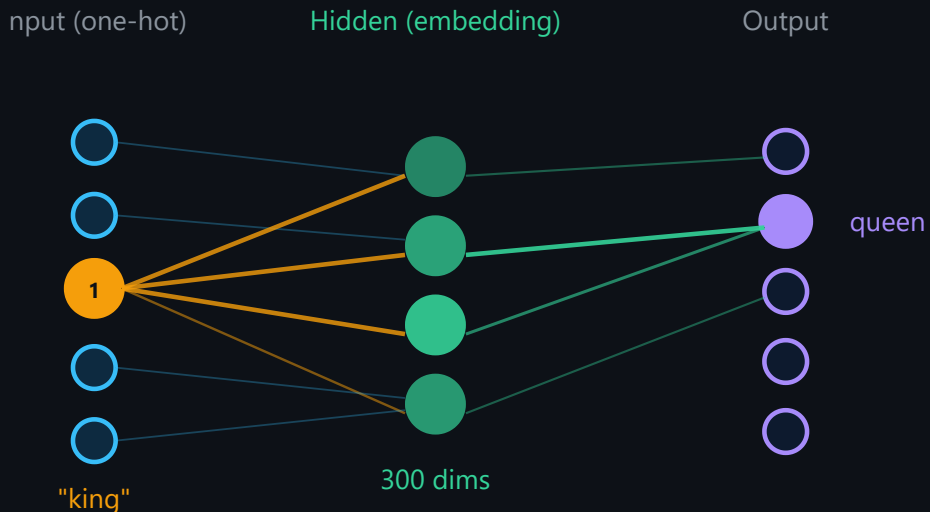


LIMITATIONS

- ⚠ No word order or syntax awareness
- ⚠ Synonyms treated as completely different words
- ⚠ No semantic relationships encoded
- ⚠ "bank" (river) = "bank" (finance) — no context

Key Innovation

Train a shallow neural network to predict context words. The **hidden layer weights become dense, meaningful word embeddings**.



Two Architectures

CBOW: Predict target from context

"__ rules the land" → "king"

Skip-gram: Predict context from target

"king" → ["rules", "land", "castle"]

✓ Breakthrough

First dense, semantically meaningful embeddings. Enables word arithmetic!

LIMITATIONS

- ⚠ Static — one vector per word always
- ⚠ "bank" same vector regardless of context
- ⚠ No sentence-level understanding
- ⚠ Out-of-vocabulary words unsupported

Key Innovation

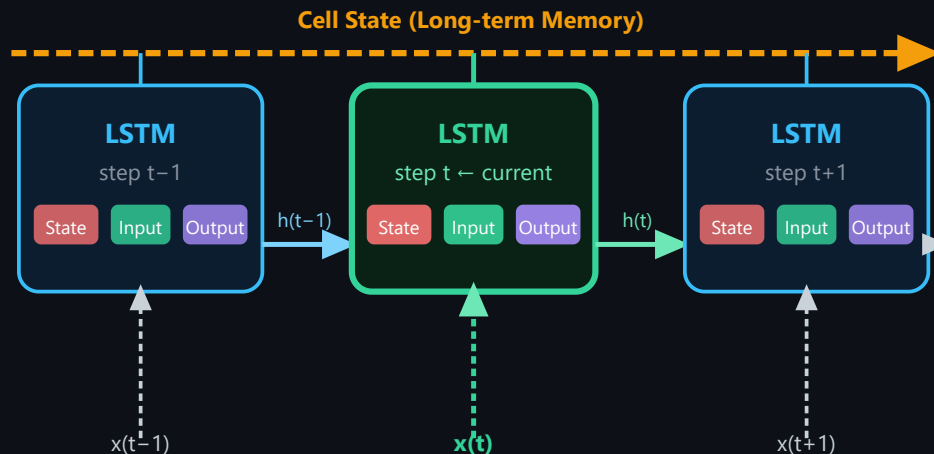
Process text **sequentially**, maintaining a memory cell that selectively **remembers or forgets** information over long sequences.

Three Gates

- **State** — what to keep in memory
- **Input** — what new info to store
- **Output** — what to output as hidden state

✓ Improvement

Handles long-range dependencies. Context-aware representations. SOTA for sequences before Transformers.



LIMITATIONS

- ⚠ Sequential — cannot be parallelized (slow!)
- ⚠ Vanishing gradient for very long sequences
- ⚠ Limited long-range context
- ⚠ No direct word-to-word attention

Language Models

Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit,
Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

Abstract. The dominant sequence transduction models are based on complex recurrent or convolutional neural networks. We propose a new simple network architecture, the **Transformer**, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely...

NeurIPS 2017 — The paper that changed everything in NLP

The Core Task

Given a sequence of words, **predict the next word**. Simple yet powerful — training on billions of documents at scale leads to deep language understanding.

EXAMPLES

Input: "The Eiffel Tower is located in"

→ Predict: "Paris" ($p = 0.91$)

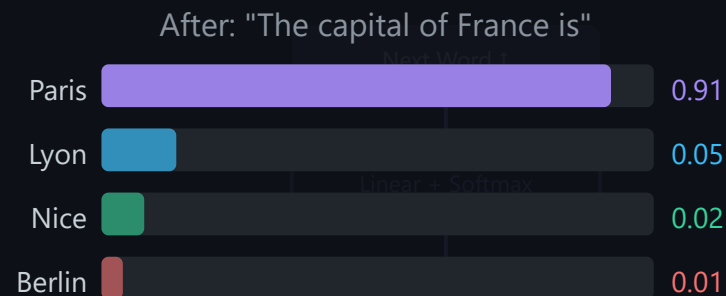
Input: "Rivers flow into the"

→ Predict: "sea" / "ocean" / "lake"



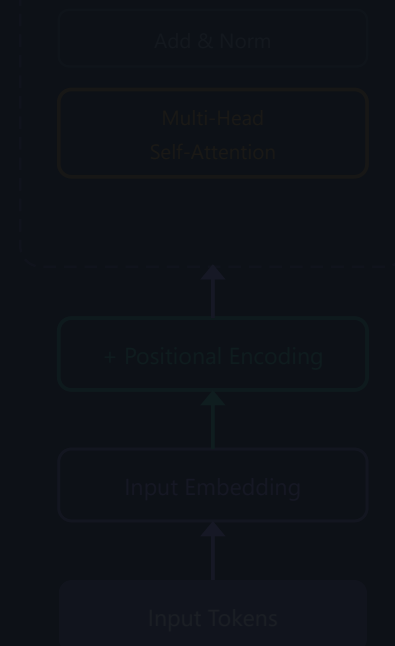
Why It Works

To predict next words accurately, the model must learn grammar, facts, geography, and world knowledge from training data.



Model learns $P(w_t | w_1, w_2, \dots, w_{t-1})$
via training on massive text corpora

$$P(w_t | w_1, w_2, \dots, w_{t-1})$$



Why Transformers?

Unlike LSTMs, Transformers process **all tokens in parallel** and relate any two positions directly via attention — no sequential bottleneck.

⚡ Key Advantage

Every token attends to every other token **simultaneously**. Long-range dependencies solved in $O(1)$ steps vs $O(n)$ for RNNs.

KEY COMPONENTS (DECODER-ONLY)

- 1 **Tokenization** — raw text → token IDs
- 2 **Input Embedding** — token IDs → dense vectors
- 3 **Positional Encoding** — inject position info
- 4 **Multi-Head Self-Attention** — relate all tokens simultaneously
- 5 **Linear Layers** — project hidden state to vocabulary probabilities

Scale

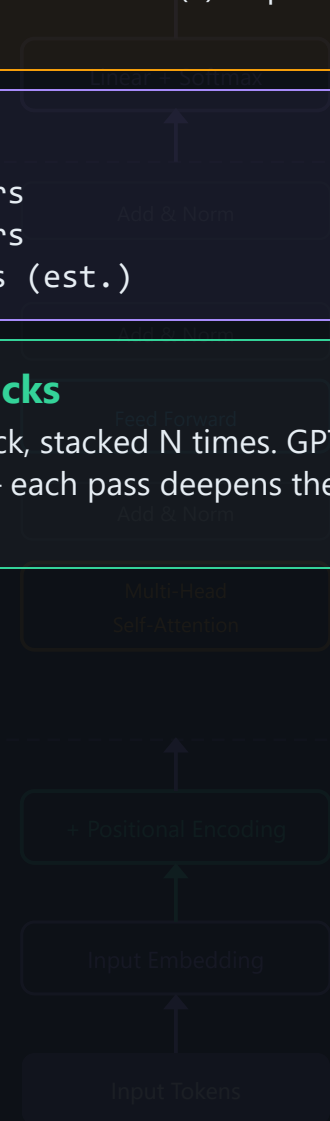
GPT-2: 1.5B parameters

GPT-3: 175B parameters

GPT-4: ~1T parameters (est.)

N × Transformer Blocks

Steps 4 & 5 form one block, stacked N times. GPT-2: 12 layers, GPT-3: 96 layers — each pass deepens the model's language understanding.



The Problem

The Transformer processes all tokens **simultaneously**, with no built-in notion of order. Without position info, "dog bites man" = "man bites dog" to the model!

The Solution

Add a **positional encoding vector** to each token embedding, giving the model unique position information for each slot.

$$\begin{aligned} PE(pos, 2i) &= \sin(pos/10000^{(2i/d)}) \\ PE(pos, 2i+1) &= \cos(pos/10000^{(2i/d)}) \end{aligned}$$

pos=position, i=dim index, d=embedding dim



Combined input at position i:

$$z_i = E(token_i) + PE(i)$$

$E(token_i)$ → learned d-dim embedding

$PE(i)$ → fixed sin/cos vector

z_i → fed into Transformer



Three Projections

Q Query — "What am I looking for?"

K Key — "What do I offer?"

V Value — "What do I pass on?"

THE FORMULA

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

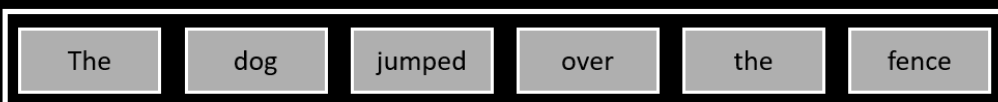
Scale $\div \sqrt{d_k}$ prevents gradient issues

Key Insight

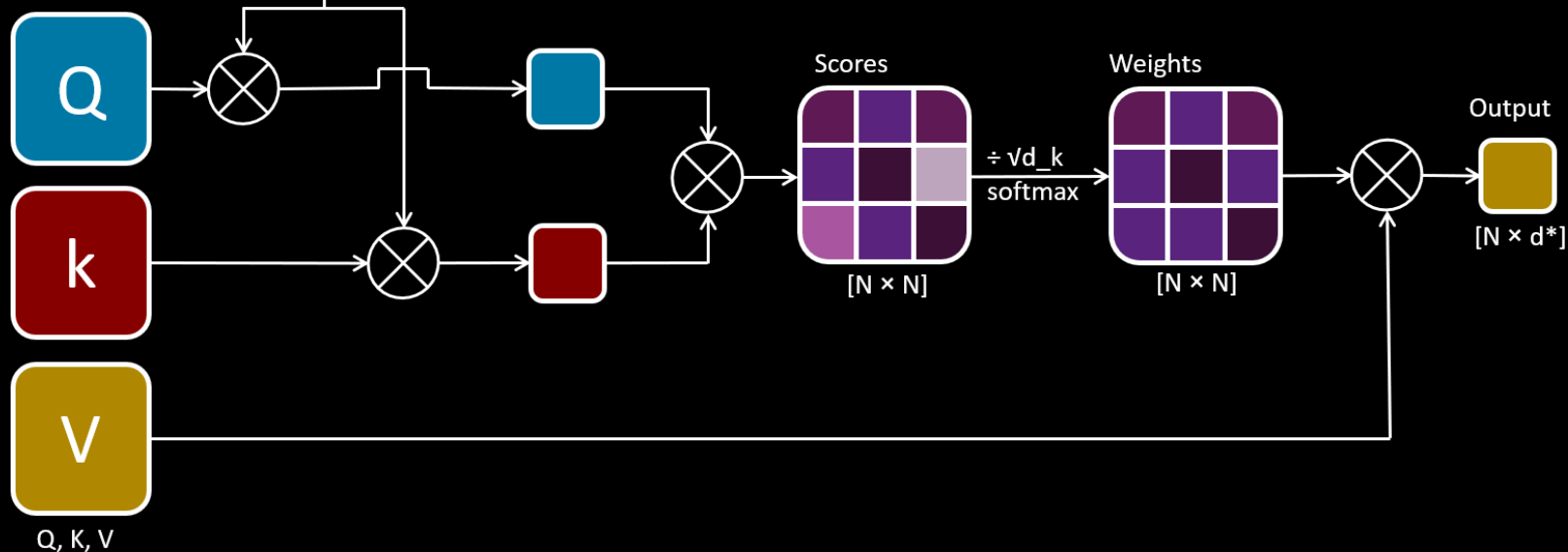
Every token can **directly attend** to every other token in one step — no sequential bottleneck!

Linear + Softmax

Input X



$[N \times d]$

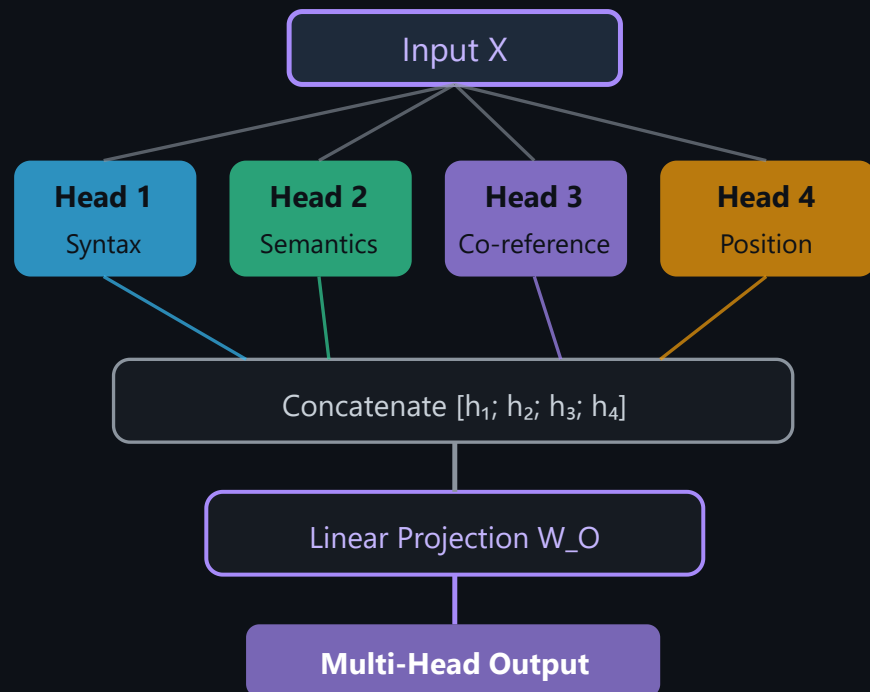


Input Embedding

Input Tokens

The Big Idea

Run **h** **parallel attention heads**, each with different learned projections. Each head captures *different types of relationships* simultaneously.



What Each Head Learns

(these are examples — actual heads are emergent)

- **Head 1:** Syntactic relations (subject-verb)
- **Head 2:** Semantic similarity
- **Head 3:** Coreference (it → entity)
- **Head 4:** Positional / local patterns

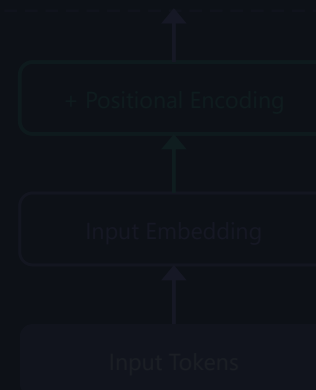
The Formula

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

⚡ Why It Works

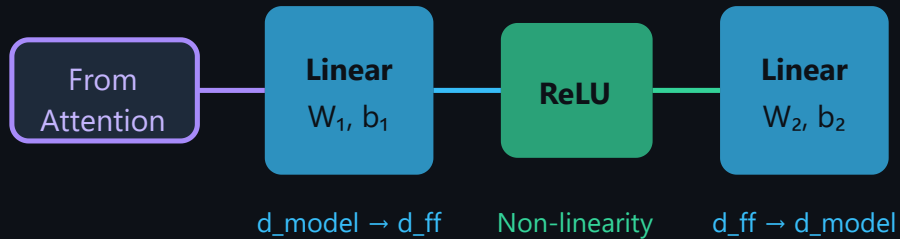
Multiple heads = multiple "perspectives" on the same input. Richer, more nuanced representation.



Feed-Forward Network (FFN)

After attention, each token's representation passes through a **position-wise FFN** — two linear layers with ReLU activation.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$



Layer Normalization

Applied after attention and FFN. **Stabilizes training** and enables very deep networks.

$$\text{LayerNorm}(x) = \gamma \cdot (x - \mu) / \sigma + \beta$$

Residual Connections

Each sub-layer adds its input to its output:

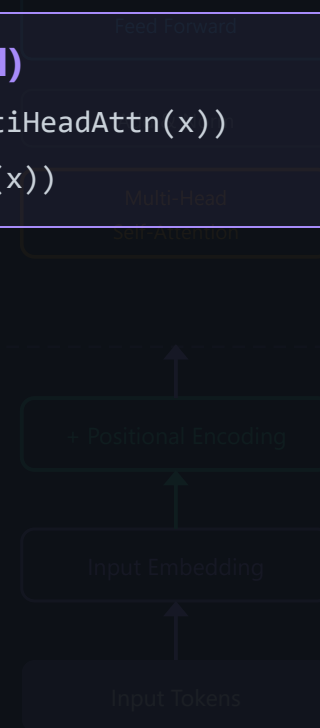
$$\text{output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

→ Prevents vanishing gradients, enables depth

Complete Block (×N)

$$x = \text{LayerNorm}(x + \text{MultiHeadAttn}(x))$$

$$x = \text{LayerNorm}(x + \text{FFN}(x))$$



Part 1: NLP Basics

- ✦ NLP = understanding human language
- ✦ Language is ambiguous & contextual
- ✦ Latent spaces encode meaning as geometry
- ✦ Cosine similarity measures direction
- ✦ Word arithmetic works!
(king – man + woman ≈ queen)

Part 2: Text Representation

- ✦ BoW: simple word counts (sparse)
- ✦ TF-IDF: weighted word importance
- ✦ Word2Vec: dense neural embeddings
- ✦ LSTM: sequential context modeling
- ✦ All limited by context or scale

Part 3: Transformers

- ✦ Task: next word prediction
- ✦ Positional encoding for word order
- ✦ Self-Attention: Q, K, V mechanism
- ✦ Multi-Head: parallel perspectives
- ✦ FFN + LayerNorm = complete block

GeoAI Connection

Language Models unlock: **geographic entity extraction**, **place descriptions**, **spatial reasoning from text**, and **geo-question answering** — all powered by the architectures we explored today.

